

The problem:

Tools without guidance

Introduction

From individual tools to coordinated systems

Your journey:

Part 1

Understood tools fundamentally and
built first tool-enabled agent

Part 2

Used built-in Google Search and code execution tools

Part 3

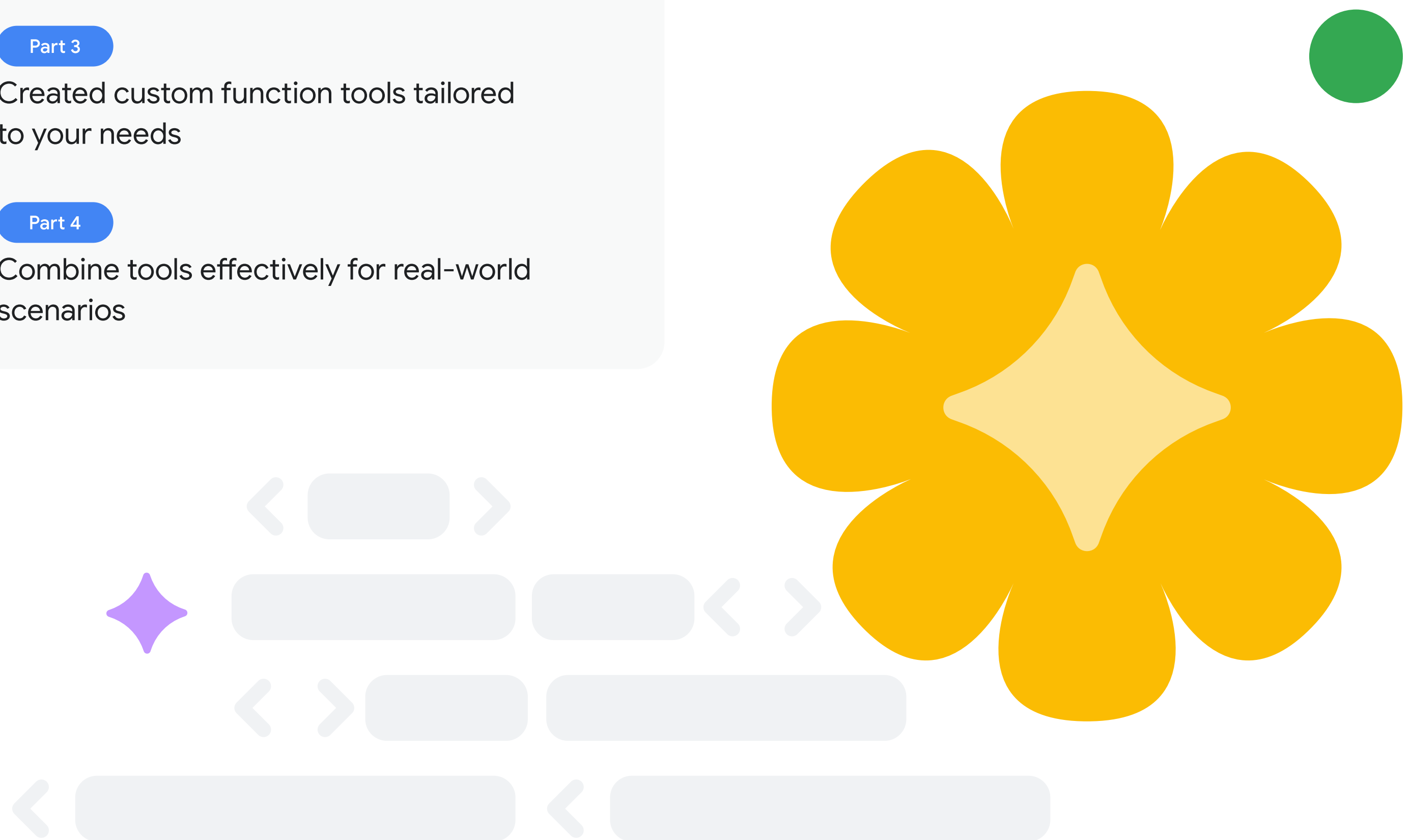
Created custom function tools tailored to your needs

Part 4

Combine tools effectively for real-world scenarios

The shift:

Tools alone aren't enough—you need strategic instructions to guide agents in using them effectively.



The problem:

Tools without guidance

You have powerful tools, but without proper instruction design, agents struggle:

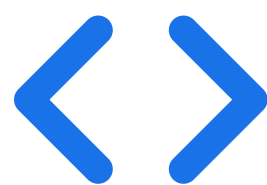
```
Python
from google.adk.agents import
LlmAgent

# You have multiple tools...
def check_order_status(order_id:
str) -> dict:
    """Check the status of an
order."""
    # Implementation
    pass

def process_refund(order_id: str,
amount: float) -> dict:
    """Process a refund for an
order."""
    # Implementation
    pass

def lookup_customer(email: str) ->
dict:
    """Look up customer
information."""
    # Implementation
    pass

# But with vague instructions...
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="You are a customer
service agent.", # Too vague!
    tools=[check_order_status,
process_refund, lookup_customer]
)
```



Issues with generic solutions:

- ❌ **Wrong tool selection:** Agent uses `process_refund` before checking if order exists
- ❌ **Poor error handling:** Agent doesn't know what to do when tool returns error
- ❌ **Wrong order:** Agent calls tools in illogical sequence
- ❌ **Ignored results:** Agent doesn't properly use tool output in response
- ❌ **No escalation path:** Agent can't handle situations beyond tool capabilities

The root problem

Tools need strategic instructions to guide when, how, and in what order to use them.

